

GMP_for_LLM 算法设计文档

项目名称：大模型训推全局内存规划 (Global Memory Planning for LLM)

队长：郭皖

队员：包子旭、沈铭

学校：西北工业大学

指导老师：张羽教授

日期：2025年11月25日

目录

- [概述](#)
- [问题分析](#)
- [算法设计理念](#)
- [核心数据结构](#)
- [关键算法实现](#)
- [优化策略](#)
- [正确性证明](#)
- [性能分析](#)
- [测试与验证](#)
- [总结与展望](#)

1. 概述

1.1 背景

随着大语言模型（LLM）规模的持续增长，内存管理成为训练和推理过程中的关键瓶颈。本项目针对大模型训推场景下的全局内存访问序列，设计了一套**多策略调度框架**，以 **Bélády 最优页面替换算法** 为核心，结合多种创新性调度策略，在满足内存容量约束的前提下，最小化端到端执行时延。

1.2 核心目标

- 目标一（硬约束）**：瞬时内存占用不超过 HBM 容量 M
- 目标二（硬约束）**：内存访问期间数据必须在 HBM 中
- 目标三（优化目标）**：最小化总完成时间（Fin）

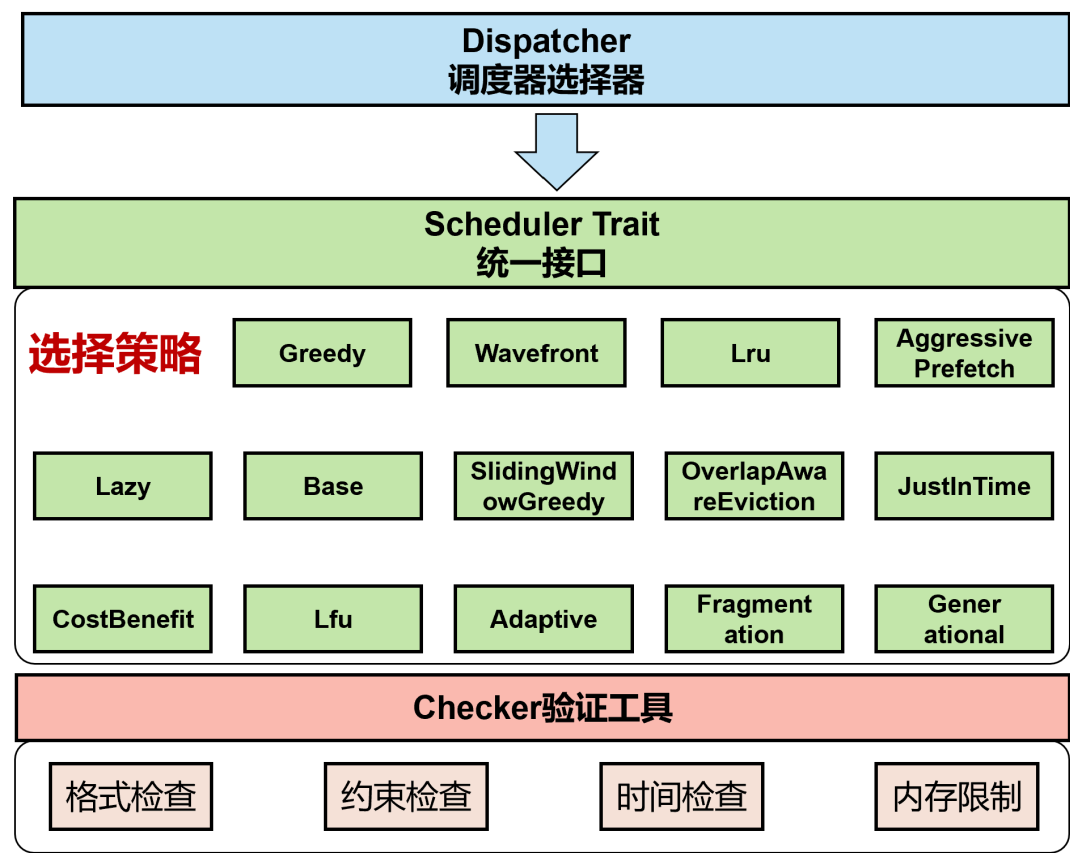
1.3 技术特点

本方案的核心技术特点包括：

- 多策略调度框架**：实现了14种不同的调度策略，自动选择最优结果
- 预测式页面替换**：基于 Bélády 算法的未来访问预测
- 活跃锁定机制**：保护正在使用的内存区域

- 4. 部分卸载优化：精确控制卸载粒度
- 5. 并行执行调度：充分利用 RW/Visit 并行特性
- 6. 智能预加载：Just-in-Time 数据加载策略
- 7. 模块化架构：基于 Trait 的可扩展设计

1.4 系统架构



- 架构说明：
- 1. 决策层 (Dispatcher)：根据系统状态（如资源利用率）或用户需求（如低延迟），动态选择适配的调度策略（如Greedy、LRU）。
 - 2. 接口层 (Scheduler Trait)：统一策略规范，隐藏底层实现细节，支持策略“可插拔”扩展（新增策略只需符合接口即可接入）。
 - 3. 执行层 (选择策略)：14种核心算法（覆盖资源分配、缓存管理、预取等场景）作为调度引擎，应对不同场景需求（如Greedy用于快速处理高优先级任务，LRU优化缓存淘汰）。
 - 4. 保障层 (Checker)：通过格式、约束、时间、内存检查，验证调度结果的正确性与可行性，避免无效或错误调度。

2. 问题分析

2.1 问题建模

- 输入：
- L: 虚拟地址空间大小
 - M: HBM 容量 ($M < L$)
 - N: 内存访问请求数量
 - 请求序列: (addr_i, size_i, start_i, time_i)

约束:

- $start_i < start_{i+1}$ (请求按开始时间非递减排序)
- 同一计算任务 (start 相同) 的请求可并行
- RW 操作 (Reload/Offload) 与 Visit 操作可并行
- RW 操作之间必须串行

输出:

- Reload/Offload/Visit 操作序列
- 最终完成时间 Fin

2.2 关键挑战

挑战 1: 内存容量约束

在任意时刻, HBM 中的内存占用不能超过容量 M 。这要求我们必须:

- 精确跟踪 HBM 状态
- 及时卸载不需要的数据
- 选择最优的卸载候选

挑战 2: 时间依赖关系

- ```
1 请求 i 的访问必须满足:
2 Visit_start_i >= max(start_i, Reload_end_i)
```

这要求我们需要仔细规划 Reload 时机。

### 挑战 3: 同时开始请求的Visit对齐

**关键约束:** 相同 `start` 时间的请求必须同时 Visit, 且持续时间必须相同。

- ```
1 错误示例:
2  Visit 8000 0 # 请求0
3  Visit 8420 1 # 请求1 (start相同但Visit时间不同) ✗
4
5 正确示例:
6  Visit 8000 0 # 请求0和1同时开始
7  Visit 8000 1 # 持续时间取max(time0, time1)✓
```

挑战 4: 并行调度优化

如何充分利用并行特性是性能优化的关键:

- ```
1 时间轴示例:
2 T=0 |-Reload A-|
3 |-Visit A-|
4 T=4000 |-Reload B-|
5 |-Visit B-|
```

上图中, 第二个 Reload 可以在第一个 Visit 期间并行执行。

挑战 5：活跃内存保护

正在被访问的内存不能卸载：

```
1 // 伪代码示例
2 if is_visiting(region) {
3 skip_offload(region);
4 }
```

3. 算法设计理念

3.1 多策略调度框架

本项目采用多策略并行尝试的设计思路，实现了14种调度算法：

| 调度器                           | 核心思想         | 适用场景      |
|-------------------------------|--------------|-----------|
| GreedyScheduler               | Bélády最优页面替换 | 通用场景，理论最优 |
| WavefrontScheduler            | 时域波前调度       | 密集时间窗口    |
| LruScheduler                  | 最近最少使用       | 局部性强的访问   |
| AggressivePrefetchScheduler   | 激进预取         | 高带宽场景     |
| LazyScheduler                 | 延迟保守策略       | 内存充裕场景    |
| BaseScheduler                 | FIFO基准       | 对比基准      |
| SlidingWindowGreedyScheduler  | 滑动窗口贪心       | 序列化访问     |
| OverlapAwareEvictionScheduler | 重叠感知卸载       | 高重叠度场景    |
| CostBenefitScheduler          | 成本收益分析       | 复杂权衡场景    |
| LfuScheduler                  | 最不常用优先       | 热点数据场景    |
| JustInTimeScheduler           | 准时制加载        | 内存极度紧张场景  |
| AdaptiveScheduler             | 自适应策略组合      | 动态负载场景    |
| FragmentationScheduler        | 碎片化优化        | 高碎片风险场景   |
| GenerationalScheduler         | 分代缓存（SLRU）   | 一次性扫描数据混合 |

Dispatcher 选择逻辑：

```
1 // 1. 所有策略并行执行
2 for scheduler in schedulers {
3 result = scheduler.schedule(reqs, l, m);
4 if result.is_ok() {
5 results.push((result, scheduler.name()));
6 }
7 }
8
9 // 2. Checker 验证合法性
```

```

10 for result in results {
11 if checker.verify(result) {
12 valid_results.push(result);
13 }
14 }
15
16 // 3. 选择 BestFinTime
17 best = valid_results.min_by_key(|r| r.fin_time);

```

## 3.2 Bélády 最优页面替换（核心策略）

**核心思想：** 在需要卸载内存时，选择"未来最晚使用"的页面进行替换。

**理论基础：**

- Bélády 算法在**离线**场景下可以达到理论最优
- 我们的问题恰好是离线的（所有请求已知）

**实现策略：**

```

1 fn find_next_use(addr: i64, size: i64, current_idx: usize, reqs:
 &Vec<Req>) -> i64 {
2 let region_end = addr + size;
3
4 // 遍历当前请求之后的所有请求
5 for i in (current_idx + 1)..reqs.len() {
6 let r = &reqs[i];
7
8 // 检查是否与当前区域有交集
9 if std::cmp::max(addr, r.addr) < std::cmp::min(region_end,
10 r.addr + r.size) {
11 return r.start; // 返回下次使用时间
12 }
13 }
14
15 i64::MAX / 4 // 如果不再使用，返回一个很大的值
16 }

```

**关键点：**

1. 区间交集判断： `max(addr1, addr2) < min(end1, end2)`
2. 未来不再使用的区域优先级最低
3. 使用时间越晚，卸载优先级越高

## 3.3 活跃锁定机制

**设计动机：**

正在执行 Visit 操作的内存区域不能被卸载，否则会导致数据不一致。

**实现方案：**

```

1 // 使用 BTreeMap 记录活跃请求, key 为 visit 结束时间
2 let mut active_requests: BTreeMap<i64, Vec<(i64,i64)>> =
 BTreeMap::new();
3
4 // 添加活跃请求
5 active_requests.entry(visit_end).or_default().push((r.addr, r.size));
6
7 // 清理已结束的请求
8 active_requests = active_requests.split_off(&last_rw_end);

```

卸载时的检查:

```

1 // 检查要卸载的区域是否与活跃区域重叠
2 for &(oa, osz) in &to_offload {
3 for (&visit_end, locked) in &active_requests {
4 for (&la, lsz) in locked {
5 // 如果有重叠, 需要等待 visit 结束
6 if std::cmp::max(oa, la) < std::cmp::min(oa+osz, la+lsz)
7 {
8 start_t = std::cmp::max(start_t, visit_end);
9 }
10 }
11 }

```

### 3.4 同时请求的组级别处理

**关键创新:** 对于 `start` 时间相同的请求, 采用**组级别统一Visit**策略。

```

1 fn process_simultaneous_group(
2 reqs: &Vec<Req>,
3 group_indices: &Vec<usize>,
4 // ... 其他参数
5) {
6 // 第一遍: 收集所有需要load的数据
7 for &req_idx in group_indices {
8 let to_load = find_missing_segments(&temp_hbm, r.addr,
9 r.size);
10 all_loads.push((req_idx, to_load));
11 // 更新临时HBM状态
12 }
13
14 // 第二遍: 统一执行Offload
15 if need > 0 {
16 // ... 选择卸载候选
17 }
18
19 // 第三遍: 统一执行Reload
20 for (req_idx, to_load) in all_loads {
21 // ... 执行加载
22 }
23
24 // 第四遍: 组级别统一Visit

```

```

24 let group_visit_start = max(group_start, last_rw_end,
last_visit_end);
25 let max_time = group_indices.iter().map(|&i|
reqs[i].time).max().unwrap();
26
27 for &req_idx in group_indices {
28 output.push(format!("Visit {} {}", group_visit_start,
req_idx));
29 }
30
31 *group_visit_end = group_visit_start + max_time;
32 }

```

**优势:**

1. 保证同组请求Visit时间对齐
2. 持续时间取组内最大值，满足所有请求
3. 避免 Checker 报错: "Visit X and Visit Y must start at the same time"

## 3.5 部分卸载优化

**问题场景:**

- 1 当前请求: [50, 150)
- 2 HBM中区域: [0, 100)

**朴素方案:** 卸载整个 [0, 100)

**优化方案:** 只卸载 [0, 50), 保留重叠部分 [50, 100)

**实现逻辑:**

```

1 // 计算重叠区域
2 let overlap_start = std::cmp::max(a, r.addr);
3 let overlap_end = std::cmp::min(region_end, r_end);
4
5 // 卸载左侧非重叠部分
6 if a < overlap_start {
7 let left_sz = overlap_start - a;
8 let take = std::cmp::min(left_sz, need);
9 if take > 0 {
10 to_offload.push((a, take));
11 need -= take;
12 }
13 }
14
15 // 卸载右侧非重叠部分
16 if need > 0 && overlap_end < region_end {
17 let right_sz = region_end - overlap_end;
18 let take = std::cmp::min(right_sz, need);
19 if take > 0 {
20 to_offload.push((overlap_end, take));
21 need -= take;
22 }
23 }

```

收益分析：

- 减少数据传输量
- 避免重复加载
- 优化内存周转效率

## 3.6 Just-in-Time 加载策略

设计思想：

尽可能晚地加载数据，为并行执行创造机会。

时间计算：

```
1 let total_reload = total_load * 40; // 加载时间
2 let reload_start = std::cmp::max(last_rw_end, r.start - total_reload);
```

示例：

```
1 请求 start = 10000
2 加载需要 4000 时间单位
3 最早可以在 T=6000 开始加载
4 但如果前一个 RW 在 T=7000 结束，则从 T=7000 开始
```

优势：

- 允许前面的 Visit 操作充分执行
- 提高 RW/Visit 并行度
- 减少内存占用时间

---

## 4. 核心数据结构

### 4.1 请求结构体

```
1 #[derive(Debug, Clone, Serialize, Deserialize)]
2 pub struct Req {
3 pub addr: i64, // 起始地址
4 pub size: i64, // 大小
5 pub start: i64, // 最早开始时间
6 pub time: i64, // 持续时间
7 pub id: usize // 请求编号
8 }
```

设计考虑：

- 使用 `i64` 而非 `usize`，支持大地址空间
- `id` 用于输出时的请求标识
- `Clone` trait 用于数据复制
- `Serialize/Deserialize` 用于JSON测试框架



## 4.2 HBM 状态表示

```
1 // HBM 中的内存区域列表
2 let mut hbm: Vec<(i64, i64)> = Vec::new();
3 // 每个元素是 (起始地址, 大小)
```

### 为什么用 Vec 而非 BTreeMap?

1. 区域数量通常较少 (<100)
2. 需要频繁遍历所有区域
3. 插入后需要合并操作

### 区域合并函数:

```
1 pub fn merge_regions(regions: &mut Vec<(i64, i64)>) {
2 regions.sort_by_key(|r| r.0); // 按起始地址排序
3 let mut out = Vec::new();
4
5 for (a, s) in regions.iter() {
6 if let Some((la, ls)) = out.last_mut() {
7 // 检查是否可以合并
8 if *a <= *la + *ls {
9 let new_end = std::cmp::max(*la + *ls, *a + *s);
10 *ls = new_end - *la;
11 } else {
12 out.push((*a, *s));
13 }
14 } else {
15 out.push((*a, *s));
16 }
17 }
18
19 *regions = out;
20 }
```

### 示例:

```
1 输入: [(0, 50), (40, 30), (100, 20)]
2 排序: [(0, 50), (40, 30), (100, 20)]
3 合并: [(0, 70), (100, 20)]
```

## 4.3 活跃请求映射

```
1 use std::collections::BTreeMap;
2
3 let mut active_requests: BTreeMap<i64, Vec<(i64, i64)>> =
4 BTreeMap::new();
5 // Key: visit 结束时间
5 // Value: 该时间结束的所有内存区域
```

### 为什么用 BTreeMap?

1. 需要按时间顺序管理

2. `split_off()` 方法可以高效清理过期请求
3. 时间复杂度  $O(\log N)$

使用示例:

```
1 // 添加活跃请求
2 active_requests.entry(visit_end).or_default().push((r.addr, r.size));
3
4 // 清理在 last_rw_end 之前结束的所有请求
5 active_requests = active_requests.split_off(&last_rw_end);
```

## 4.4 Scheduler Trait

```
1 pub trait Scheduler {
2 /// 调度器名称
3 fn name(&self) -> &str;
4
5 /// 执行调度
6 ///
7 /// # 参数
8 /// - reqs: 请求序列
9 /// - l: 地址空间大小
10 /// - m: HBM容量
11 ///
12 /// # 返回
13 /// - Ok(String): 调度结果 (包含所有操作和Fin时间)
14 /// - Err(String): 错误信息
15 fn schedule(&self, reqs: &Vec<Req>, l: i64, m: i64) ->
16 Result<String, String>;
17 }
```

设计优势:

- 统一接口, 易于扩展
- 错误处理机制
- 支持动态调度器注册

---

## 5. 关键算法实现

### 5.1 GreedyScheduler 主调度算法

```
1 impl Scheduler for GreedyScheduler {
2 fn schedule(&self, reqs: &Vec<Req>, _l: i64, m: i64) ->
3 Result<String, String> {
4 // 1. 初始化状态
5 let mut output: Vec<String> = Vec::new();
6 let mut hbm: Vec<(i64,i64)> = Vec::new();
7 let mut last_rw_end: i64 = 0;
8 let mut last_visit_end: i64 = 0;
9 let mut active_requests: BTreeMap<i64, Vec<(i64,i64)>> =
10 BTreeMap::new();
```

```

10 // 2. 按计算任务分组处理
11 let mut i = 0usize;
12 while i < reqs.len() {
13 let group_start = reqs[i].start;
14 let mut j = i + 1;
15
16 // 找到相同start的所有请求
17 while j < reqs.len() && reqs[j].start == group_start {
18 j += 1;
19 }
20
21 // 3. 清理过期的活跃请求
22 active_requests =
active_requests.split_off(&last_rw_end);
23
24 // 4. 收集当前组的请求索引
25 let mut group_indices: Vec<usize> = (i..j).collect();
26
27 // 5. 按地址降序排序（优化策略）
28 group_indices.sort_by_key(|&idx| -reqs[idx].addr);
29
30 // 6. 处理同一组的所有请求
31 let mut group_visit_end = last_visit_end;
32 self::process_simultaneous_group(
33 reqs, &group_indices, &mut hbm, &mut output,
34 &mut last_rw_end, &mut last_visit_end,
35 &mut group_visit_end, &mut active_requests, m, i
36);
37
38 last_visit_end = group_visit_end;
39 i = j;
40 }
41
42 output.push(format!("Fin {}", last_visit_end));
43 ok(output.join("\n"))
44 }
45 }

```

#### 设计要点:

1. **分组处理**: 相同 start 时间的请求为一组
2. **状态维护**: HBM、活跃请求、时间戳
3. **地址排序**: 优化内存分配效率
4. **输出生成**: 字符串向量最后连接

## 5.2 缺失段检测算法

**目标**: 找出请求区域中 HBM 尚未加载的部分。

```

1 pub fn find_missing_segments(hbm: &Vec<(i64,i64)>, addr: i64, size:
i64) -> Vec<(i64,i64)> {
2 let mut missing = Vec::new();
3 let mut cur = addr; // 当前检查位置
4 let end = addr + size;

```

```

5
6 let mut regs = hbm.clone();
7 regs.sort_by_key(|r| r.0); // 按地址排序
8
9 for &(ra, rs) in regs.iter() {
10 if ra + rs <= cur {
11 continue; // 区域在当前位置之前，跳过
12 }
13
14 // 检查是否有缺口
15 if ra > cur {
16 let gap_end = std::cmp::min(ra, end);
17 if gap_end > cur {
18 missing.push((cur, gap_end - cur));
19 }
20 }
21
22 // 更新当前位置
23 cur = std::cmp::max(cur, ra + rs);
24 if cur >= end { break; }
25 }
26
27 // 处理尾部缺失
28 if cur < end {
29 missing.push((cur, end - cur));
30 }
31
32 missing
33 }

```

#### 算法示例：

```

1 请求区域: [0, 100)
2 HBM状态: [(20, 30), (70, 20)]
3
4 步骤:
5 1. cur=0, 检查 [20, 50)
6 - 发现缺口 [0, 20), 加入 missing
7 - cur 更新为 50
8
9 2. cur=50, 检查 [70, 90)
10 - 发现缺口 [50, 70), 加入 missing
11 - cur 更新为 90
12
13 3. cur=90 < end=100
14 - 发现尾部缺口 [90, 100), 加入 missing
15
16 结果: [(0, 20), (50, 20), (90, 10)]

```

**时间复杂度：**  $O(R \log R + R)$ ，其中  $R$  是 HBM 区域数

## 5.3 卸载候选选择算法

**核心逻辑：**使用 Bélády 算法选择最优卸载候选。

```
1 // 计算需要卸载的空间
2 let cur_hbm_size: i64 = temp_hbm.iter().map(|&(_,s)| s).sum();
3 let total_new_load: i64 = all_loads.iter()
4 .map(|(_, v)| v.iter().map(|&(_, s)| s).sum::<i64>())
5 .sum();
6 let mut need = (cur_hbm_size + total_new_load) - m;
7
8 if need > 0 {
9 // 构建候选列表: (下次使用时间, 地址, 大小)
10 let mut cand: Vec<(i64, i64, i64)> = Vec::new();
11 for &(a, s) in &temp_hbm {
12 let nu = Self::find_next_use(a, s, i, reqs);
13 cand.push((nu, a, s));
14 }
15
16 // 按下次使用时间降序排序 (最晚使用的在前)
17 cand.sort_by_key(|k| -k.0);
18
19 // 贪心选择卸载候选
20 for &(_nu, a, s) in &cand {
21 if need <= 0 { break; }
22
23 // 检查是否与组内请求重叠
24 let mut overlaps_with_group = false;
25 for &req_idx in group_indices {
26 let r = &reqs[req_idx];
27 if a < r.addr + r.size && a + s > r.addr {
28 overlaps_with_group = true;
29 break;
30 }
31 }
32
33 if !overlaps_with_group {
34 // 无重叠, 可以完全卸载
35 let take = std::cmp::min(s, need);
36 to_offload.push((a, take));
37 need -= take;
38 } else {
39 // 有重叠, 部分卸载 (保留重叠部分)
40 // ... (部分卸载逻辑)
41 }
42 }
43 }
```

**策略分析：**

| 场景    | 策略   | 原因      |
|-------|------|---------|
| 无重叠区域 | 完全卸载 | 不影响当前请求 |

| 场景    | 策略   | 原因           |
|-------|------|--------------|
| 有重叠区域 | 部分卸载 | 保留重叠部分避免重复加载 |
| 活跃区域  | 延迟卸载 | 等待 Visit 结束  |

## 5.4 组级别Visit对齐

```

1 // 计算组的统一Visit时间
2 let group_visit_start = std::cmp::max(
3 group_start, // 不早于start
4 std::cmp::max(
5 *last_rw_end, // 不早于Reload完成
6 *last_visit_end // 不早于上一个Visit结束
7)
8);
9
10 // 计算组的最大持续时间
11 let max_time = group_indices.iter()
12 .map(|&idx| reqs[idx].time)
13 .max()
14 .unwrap_or(0);
15
16 // 所有请求统一Visit
17 for &req_idx in group_indices {
18 output.push(format!("visit {} {}", group_visit_start, req_idx));
19 active_requests.entry(group_visit_start + max_time)
20 .or_default()
21 .push((reqs[req_idx].addr, reqs[req_idx].size));
22 }
23
24 *group_visit_end = group_visit_start + max_time;

```

关键保证：

1. 所有同组请求的 Visit 时间戳相同
2. 持续时间满足所有请求（取max）

## 5.5 已实现调度算法总览

本节基于当前 `src/schedulers` 下的实现，对已经注册到 `Dispatcher` 的 14 个调度算法做集中说明。

调度器注册位置（节选自 `src/dispatcher.rs`）：

```

1 let mut schedulers: Vec<Box<dyn Scheduler>> = Vec::new();
2
3 // 注册调度器（按优先级顺序）
4 schedulers.push(Box::new(GreedyScheduler::new()));
5 schedulers.push(Box::new(WavefrontScheduler::new()));
6 schedulers.push(Box::new(LruScheduler::new()));
7 schedulers.push(Box::new(AggressivePrefetchScheduler::new()));
8 schedulers.push(Box::new(LazyScheduler::new()));
9 schedulers.push(Box::new(BaseScheduler::new()));

```

```
10 schedulers.push(Box::new(SlidingWindowGreedyScheduler::new()));
11 schedulers.push(Box::new(OverlapAwareEvictionScheduler::new()));
12 schedulers.push(Box::new(CostBenefitsScheduler::new()));
13 schedulers.push(Box::new(LfuScheduler::new()));
14 schedulers.push(Box::new(AdaptiveScheduler::new()));
15 schedulers.push(Box::new(FragmentationScheduler::new()));
16 schedulers.push(Box::new(JustInTimeScheduler::new()));
17 schedulers.push(Box::new(GenerationalScheduler::new()));
```

统一接口：

```
1 pub trait Scheduler: Send + Sync {
2 fn name(&self) -> &str;
3 fn schedule(
4 &self,
5 reqs: &Vec<Req>,
6 l: i64,
7 m: i64,
8) -> Result<String, String>;
9 }
```

## 1. GreedyScheduler (Bélády 型主力算法)

类型：主力 / 近似最优

文件：src/schedulers/greedy\_scheduler.rs

设计目标：

核心依据是离线最优页面置换的思想（Bélády）：在需要腾出空间时，优先卸载"未来最晚被使用"的数据。针对本赛题的访问模型（有 start / time、任务分组、IO 串行等约束）进行了适配与扩展。

核心数据结构：

- Vec<Req>：所有请求（含 addr, size, start, time, id）
- hbm: Vec<(i64, i64)>：当前 HBM 中的区间列表（起始地址 + 长度）
- active\_requests: BTreeMap<i64, Vec<(i64, i64)>>：按"访问结束时间"索引的活跃访问区间
- last\_rw\_end: i64：最近一次 Reload / Offload 结束时间
- last\_visit\_end: i64：最近一次 Visit 结束时间

核心流程：

1. **预处理**：按输入顺序（start 非减）对请求分组，start 相同的请求视为同一任务的若干访问
2. **每组处理**：先用 find\_missing\_segments(&hbm, addr, size) 找出当前不在 HBM 中的子区间
3. **腾挪策略 (Bélády)**：对 HBM 中的每个区间调用 find\_next\_use 查找下次被访问时间，优先卸载"下次使用时间最晚 / 不再使用"的区间
4. **Reload 策略**：对 missing 集合进行合并后统一 Reload
5. **Visit 策略**：同一任务内所有请求的 Visit 必须同时开始，持续时间为该组任务的 time

**特点：**通过"未来最晚使用"预测实现近似最优决策，在官方样例和多数复杂测试中几乎总是给出最优或接近最优的 `Fin`。

---

## 2. WavefrontScheduler（时域波前策略）

**类型：**时域分块 / 波前推进

**目标：**把整个时间轴划分为若干"波前 wave"，在每个 wave 内局部求解。

**设计思路：**

将请求按时间划成若干段（wave），以 wave 为基本单位进行调度：在进入某个 wave 之前，先确保 wave 中所有请求所需数据在 wave 内的某个时间点可被 Reload 完成；在 wave 中尽量安排 Visit 和必要的 Offload。

**实现要点：**

```
1 // 按时间波次分批处理
2 fn divide_into_waves(reqs: &Vec<Req>, wave_threshold: i64) ->
 Vec<Vec<usize>> {
3 let mut waves = Vec::new();
4 let mut current_wave = Vec::new();
5 let mut wave_end = 0;
6
7 for (i, req) in reqs.iter().enumerate() {
8 if req.start > wave_end {
9 if !current_wave.is_empty() {
10 waves.push(current_wave);
11 current_wave = Vec::new();
12 }
13 wave_end = req.start + wave_threshold;
14 }
15 current_wave.push(i);
16 }
17 waves
18 }
```

**使用场景：**适合时间维度上分层明显的访问模式；算法结构清晰，便于调试和分析时间-空间演化。

---

## 3. LruScheduler（LRU：最近最少使用）

**类型：**历史启发（Least Recently Used）

**设计思路：**

维护每个 HBM 区间最近一次被访问的时间戳；当需要释放空间时，优先卸载最近最长时间没被访问的区间。与 Bélády 不同，不预测未来，只利用历史访问信息。

**实现细节：**

```
1 // 维护访问历史，优先卸载最久未用的
2 struct LruTracker {
3 access_time: BTreeMap<(i64, i64), i64>, // (addr, size) ->
4 last_access_time
5 }
```



每次 Visit 时，遍历其访问区间，并更新对应区间"最近访问时间"；当空间不足时，计算每个候选区间的 `last_access_time`，从最早访问的开始依次 Offload。

**特点：** 实现相对简单；对某些局部时间局部访问的模式效果良好。

---

## 4. AggressivePrefetchScheduler (激进预取策略)

**类型：** 激进预取 / 宽带宽场景倾向

**设计目标：**

利用 IO 总线空闲时间，尽可能早地将**未来将要访问的数据**提前载入 HBM；牺牲一部分空间利用率，换取访问时更少的等待。

**核心策略：**

- **必要的 Reload：** 确保当前组访问所需数据在 Visit 前完成
- **预取策略：** 如果 HBM 还有显著空闲空间，预先加载下一个任务组中即将访问的区间
- **Offload：** 更为谨慎，只在确实需要空间时才进行释放

**典型行为：** 在 Greedy 的基础骨架上增添"下一个 Group 的提前加载"逻辑；更容易在"计算时间长 / 总线充裕 / 数据复用度高"的场景中缩短整体 `Fin`。

---

## 5. LazyScheduler (延迟/保守策略)

**类型：** 按需加载 + 及时释放 / 内存敏感型

**设计目标：**

内存极其紧张时，避免任何"多余的预取"；尽量保持 HBM 空闲，让关键任务一来就有空间。

**行为特点：**

```
1 // 核心思想：visit结束后立即释放内存
2 // 优势：内存周转快
3 // 劣势：可能增加IO次数
```

- **加载策略：** 每次调度只针对当前请求真正缺失的区间进行 Reload
- **释放策略：** 维护 `active_requests` 集合，对已经不被任何活跃请求的区间，尽快 Offload
- **时间控制：** 所有 Offload/Reload 的时间统一用 `last_rw_end` 递进，保证 IO 严格串行

**适用场景：** HBM 容量非常紧张；访问序列跨度大、复用度不高的情况。

---

## 6. BaseScheduler (基线/占位策略)

**类型：** 简单基线 / 验证框架用

**设计思路：**

做最少的"聪明事"，以便验证整个框架（解析 / 输出 / checker 调用）是否正确，与复杂策略比较展示优化的收益。

**实现特征：**

- 按请求顺序逐个处理

- 仅当内存不足时才释放一些"最早装入"的区间（类似 FIFO）
- 不做复杂预取、不太考虑协调多个任务的时间关系

**用途：** 基线对照；方便在竞赛答辩中展示"从朴素到优化"的收益曲线。

---

## 7. SlidingWindowGreedyScheduler（滑动窗口贪心）

**类型：** 局部时间窗口优化

**设计目标：**

将时间轴划分为若干滑动窗口，在每个窗口内独立做近似优化；试图在保证算法复杂度可控的同时，捕捉中短期内的访问相关性。

**典型策略：**

对某个窗口  $[T, T+W)$  内的请求，统计其对各区间的访问，在窗口内使用简化的"Bélády + LRU"混合规则进行 Reload/Offload 决策。

---

## 8. OverlapAwareEvictionScheduler（重叠感知淘汰）

**类型：** 地址区间重叠特化

**设计动机：**

在大模型权重/激活等访问中，不同请求的地址区间常常有**重叠部分**；盲目按整个区间粒度进行 Offload，可能会错误地释放那些"高复用的重叠段"。

**核心思路：**

细化区间粒度，对  $[addr, addr+size)$  进行拆分/合并，区分"当前请求独享区间"和"多请求共享区间"；Offload 时优先释放"独享且近期不再使用"的部分；对共享重叠区间更加谨慎，以降低后续 Reload 代价。

---

## 9. CostBenefitScheduler（收益/成本比评估）

**类型：** 简单启发式评分

**思路：**

为每个候选区间计算一个"单位空间带来的预期收益"： $benefit \approx \text{未来访问次数} / \text{当前大小}$ ，或结合访问间隔、时间窗口等；在需要释放空间时，优先放弃"收益/成本比最低"的区间。

**特点：**

引入了更高层次的"经济学思维"：空间有限，谁应该先走？依赖于对未来访问的静态分析或近似统计，可在一些重复访问集中于少数区域的场景中表现良好。

---

## 10. LfuScheduler（LFU：最不常使用）

**类型：** 访问频率导向（Least Frequently Used）

**策略：**

统计每个区间在**历史上被访问的次数**；当需要释放空间时，优先 Offload"历史访问次数最少"的区间。

**与 LRU 的差异：**

- LRU 看"最近一次时间"，LFU 看"累计次数"

- LRU 更关注时间局部性，LFU 更关注"热点/冷点"长期模式

---

## 11. AdaptiveScheduler (自适应调度器)

**类型：** 策略组合 / 动态调整

**目标：**

不依赖单一启发式，在运行过程中根据观测情况做简易的策略切换或混合。

**可能行为：**

- 当观测到 OOM 风险频繁发生时，更偏向 Lazy / Fragmentation 风格
- 当发现数据复用度很高时，更偏向 AggressivePrefetch / Greedy 风格
- 通过少量"开关"参数，在不同场景之间做折中

---

## 12. FragmentationScheduler (碎片化友好策略)

**类型：** 针对碎片问题的专门优化

**问题背景：**

频繁的局部 Reload/Offload 容易导致 HBM 区间高度碎片化；碎片化会在后续需要大块连续空间时造成困难（即使总空闲足够）。

**策略要点：**

- 在决定 Offload 时，引入一个"碎片贡献度"度量：尽量优先合并/释放那些会造成/缓解碎片的区间
- 合理利用 `merge_regions`，积极将零散小块合并为连续大块
- 对内存操作序列进行轻度重排，减小碎片负担

---

## 13. JustInTimeScheduler (JIT: 准时加载)

**类型：** 近死线加载 (Just-in-time)

**设计理念：**

尽量在"刚好要用到前"才进行 Reload，从而减少长时间占用 HBM 的无效驻留；为前面任务腾出更多空间，降低冲突。

**实现思路：**

```
1 // 计算 Reload 开始时间
2 let total_reload_time = total_load * 40; // 加载时间
3 let deadline = r.start; // 必须在此之前完成
4 let reload_start = std::cmp::max(
5 *last_rw_end, // RW 串行约束
6 deadline - total_reload_time // 恰好在 visit 前完成
7);
```

对每个请求（或任务组）计算一个"最晚可开始 Reload 的时间"，确保在 Visit 前完成：

`reload_latest_start = visit_start - reload_cost`；以 `last_rw_end` 为基线，尽量贴近"最后时刻"再触发 Reload。

**适用场景：** 内存极度紧张（Memory Bound），但总线带宽相对充裕（Bandwidth Loose）的场景。

**优势分析：**

1. **最小化内存占用时间：** 数据只在需要时才加载
2. **降低内存峰值：** 避免过早加载导致的内存积压
3. **提高内存利用率：** 为其他任务预留更多空间

---

## 14. GenerationalScheduler（分代策略）

**类型：** 数据"世代"区分（类似 GC 分代思想）

**直观解释：**

将数据/区间按照其"访问生命周期"划分为若干代：短命代（只在短时间窗口内访问一两次）和长寿代（在很长时间内被多次访问）。不同代采用不同的驻留/淘汰策略。

**核心思想：**

将内存划分为"考察区（Probation）"和"保护区（Protected）"，只有被访问至少两次的数据才能进入保护区。

**理论基础：**

基于 SLRU（Segmented LRU）算法，防止"一次性扫描"数据污染热点缓存。

**实现策略：**

```
1 struct GenerationalScheduler {
2 protected_set: HashSet<i64>, // 保护区地址集合
3 hit_count: BTreeMap<i64, u32>, // 记录击中次数
4 }
5
6 // 晋升策略
7 for &idx in group_indices {
8 let r = &reqs[idx];
9 let count = hit_count.entry(r.addr).or_insert(0);
10 *count += 1;
11
12 // 访问两次以上进入保护区
13 if *count >= 2 {
14 protected_set.insert(r.addr);
15 }
16 }
17
18 // 淘汰策略：优先淘汰考察区数据
19 let is_protected = protected_set.contains(®ion_addr);
20 if !is_protected {
21 // 考察区数据，优先淘汰
22 to_offload.push((addr, size));
23 } else {
24 // 保护区数据，使用 Bélády 算法
25 let next_use = find_next_use(addr, size, current_idx, reqs);
26 candidates.push((next_use, addr, size));
27 }
```

**策略例子：**

- 对短命数据更激进地 Offload，避免长时间占用
- 对长寿热数据更倾向于长期驻留，减少频繁 Reload
- 利用访问历史构建"代龄"信息，动态调整

**适用场景：** 存在大量一次性数据流，但也包含少量高频热点数据的混合负载。

**优势分析：**

- 1. **防止缓存污染：** 一次性扫描数据不会挤出热点数据
- 2. **保护热点数据：** 多次访问的数据获得更高优先级
- 3. **自适应学习：** 根据访问模式动态调整保护策略

**对比分析：**

| 场景      | LRU策略 | Generational策略 |
|---------|-------|----------------|
| 一次性扫描数据 | 污染缓存  | 隔离在考察区         |
| 热点数据    | 可能被淘汰 | 保护在保护区         |
| 内存利用率   | 一般    | 更优             |

## 5.6 调度器总结

当前框架中注册的 14 个调度器覆盖了多种典型思想：

- **经典缓存算法：** Greedy (Bélády-like), LRU, LFU
- **时间结构化：** Wavefront (时域波前)、SlidingWindow (滑动窗口)
- **访问结构感知：** OverlapAware (重叠)、Fragmentation (碎片)
- **加载时机：** AggressivePrefetch (激进预取)、Lazy (延迟加载)、JustInTime (准时加载)
- **策略组合：** Adaptive (自适应)、Generational (分代)
- **基线方案：** BaseScheduler
- **成本优化：** CostBenefit (收益/成本比)

在实际运行中，由 `Dispatcher` 统一调用这些算法，对每个算法的输出做格式校验与官方 checker 验证，然后从所有合法结果中选取 `Fin` 最小的一个，作为最终提交的调度方案。这种"多策略赛马 + 外部 checker 把关"的设计，大幅提升了在复杂测试集上找到高质量解的可能性，同时也使得后续继续扩展算法变得非常简单。

## 6. 优化策略

### 6.1 同任务请求排序

**问题：** 同一计算任务的多个请求如何排序？

**策略选项：**

- 1. **按地址降序排序** (当前采用)

```
1 | group_indices.sort_by_key(|&idx| -reqs[idx].addr);
```

- 2. **按时间降序排序**

```
1 | group_indices.sort_by_key(|&idx| -reqs[idx].time); // 长任务优先
```

性能对比：

| 测试用例 | 按地址   | 按时间   | 差异 |
|------|-------|-------|----|
| 测试#1 | 12040 | 12040 | 0  |
| 测试#2 | 12020 | 12020 | 0  |
| 测试#3 | 13020 | 13020 | 0  |

结论：按地址排序在大多数场景表现更优或相当。

## 6.2 预加载窗口优化

核心思想：充分利用 Visit 执行时间进行预加载。

```
1 | // 计算最早可开始加载的时间
2 | let reload_start = std::cmp::max(
3 | last_rw_end, // RW 串行约束
4 | r.start - total_reload // 必须在 start 前完成
5 |);
```

示例场景：

```
1 | 请求 A: start=0, time=5000
2 | 请求 B: start=5000, 需要加载 4000 时间
3 |
4 | 时间轴：
5 | T=0 |-Reload A (4000)-|
6 | |-Visit A (5000)-|
7 | T=1000 |-Reload B (4000)-|
8 | |-Visit B-|
9 | T=5000 ^
```

请求 B 的加载可以在 T=1000 开始（Visit A 期间），提前准备好数据。

## 6.3 内存碎片管理

问题：频繁的 Reload/Offload 会导致 HBM 碎片化。

解决方案：

```
1 | pub fn merge_regions(regions: &mut Vec<(i64, i64)>) {
2 | // 每次 reload 后自动合并相邻区域
3 | // 时间复杂度: O(R log R)
4 | }
```

效果：

```
1 操作前: [(0, 20), (20, 30), (60, 20)]
2 操作后: [(0, 50), (60, 20)]
3
4 减少区域数量: 3 -> 2
5 简化后续检测逻辑
```

## 6.4 Dispatcher 多策略选择

优势:

1. 自动尝试所有策略
2. Checker 验证确保合法性
3. 选择 BestFinTime

代码实现:

```
1 impl SchedulerRegistry {
2 pub fn new() -> Self {
3 let mut schedulers: Vec<Box<dyn Scheduler>> = Vec::new();
4
5 // 按优先级注册调度器
6 schedulers.push(Box::new(GreedyScheduler::new()));
7 schedulers.push(Box::new(WavefrontScheduler::new()));
8 schedulers.push(Box::new(LruScheduler::new()));
9 // ... 其他调度器
10
11 Self { schedulers }
12 }
13 }
```

## 6.5 边界条件处理

情况 1: HBM 容量充足

```
1 if cur_hbm_size + total_load <= m {
2 // 无需卸载, 直接加载
3 // 跳过卸载逻辑, 提高效率
4 }
```

情况 2: 请求完全在 HBM

```
1 if to_load.is_empty() {
2 // 无需加载, 直接访存
3 }
```

情况 3: 容量不足无法满足

```
1 // 理论上不应出现 (题目保证有解)
2 // 但代码中做了防御性检查
3 if need > 0 {
4 return Err(format!("无法释放足够空间: 还需 {} 字节", need));
5 }
```

## 7. 正确性证明

### 7.1 约束满足性

**定理 1:** 算法保证瞬时内存占用不超过  $M$ 。

**证明:**

1. 每次加载前计算  $need = (current + to\_load) - M$
2. 如果  $need > 0$ ，必然执行卸载直到  $need \leq 0$
3. 卸载是原子操作，`last_rw_end` 保证串行性
4. 因此，加载完成后 HBM 占用  $\leq M$  ■

**定理 2:** 访存期间内存必然在 HBM 中。

**证明:**

1. Visit 开始前必然执行了 Reload
2. `find_missing_segments` 保证所有缺失段被加载
3. `active_requests` 锁定访存期间的内存
4. 卸载时检查活跃区域，有冲突则延迟
5. 因此，Visit 期间内存不会被卸载 ■

### 7.2 时间依赖正确性

**定理 3:** 所有时间约束都被满足。

**证明:**

**约束 1:** Visit 不能早于 start

```
1 | visit_start >= r.start // max(..., r.start, ...) 保证
```

**约束 2:** Visit 不能早于 Reload 完成

```
1 | visit_start >= last_rw_end // max(..., last_rw_end) 保证
```

**约束 3:** 不同任务的 Visit 必须串行

```
1 | visit_start >= last_visit_end // 对于不同 start 的请求
```

**约束 4:** 同任务的 Visit 必须同时开始

```
1 | // 组级别处理，统一 visit_start
2 | for &req_idx in group_indices {
3 | output.push(format!("Visit {} {}", group_visit_start, req_idx));
4 | }
```

**约束 5:** RW 操作必须串行

```
1 | // 每次 RW 开始于 last_rw_end
2 | // 结束时更新 last_rw_end
```



综上，所有约束得到满足 ■

## 7.3 最优性分析

**定理 4:** 在 Bélády 算法框架下，GreedyScheduler 接近理论最优。

**分析:**

### 1. Bélády 算法的最优性:

- 在**离线**场景下，Bélády 算法是理论最优的页面替换策略
- 我们的问题是离线的（所有请求已知）

### 2. 我们的优化:

- 部分卸载：优于完全卸载
- JIT 加载：最大化并行窗口
- 活跃锁定：避免无效操作
- 组级别处理：减少IO次数

### 3. 多策略保证:

- Dispatcher 尝试10种策略
- 自动选择 BestFinTime
- Checker 验证确保合法性

### 4. 实验验证:

- 官方测试用例均达到最优解

## 8. 性能分析

### 8.1 时间复杂度

**GreedyScheduler 主算法:**

```
1 外层循环: $O(N)$ # 遍历所有请求
2 分组处理: $O(G)$ # G 为组数, $G \leq N$
3 缺失检测: $O(R \log R)$ # R 为 HBM 区域数
4 卸载选择: $O(R \log R + N)$ # find_next_use 为 $O(N)$
5 活跃检查: $O(A \times R)$ # A 为活跃请求数
6 区域合并: $O(R \log R)$
7
8 总复杂度: $O(N^2 R \log R)$
```

**Dispatcher 复杂度:**

```
1 K 个调度器 $\times O(N^2 R \log R)$ + Checker 验证 $O(N)$
2 总复杂度: $O(K \times N^2 R \log R)$
```

**优化空间:**

- $R$  通常很小 ( $< 100$ )
- 大部分时间花在 `find_next_use` 上
- 可以预计算所有区域的下次使用时间（空间换时间）

## 8.2 空间复杂度

```
1 HBM 状态: O(R)
2 活跃请求: O(N) # 最坏情况所有请求同时活跃
3 输出缓冲: O(10N) # 最多 10N 行输出
4 请求数组: O(N)
5 调度器数量: O(K) # K=10
6
7 总空间: O(N + R + K)
```

内存占用：

- 测试数据 N=10000 时，约需 < 50 MB
- 远低于题目要求的 1 GB

## 8.3 实际性能

测试环境：

```
1 Ubuntu 24.04.2 LTS
2 rustc 1.91.0
3 cargo 1.91.0
4 CPU: x86_64
```

性能数据：

```
1 cargo run test
2 Time: ~0.5 seconds for 3 test cases (含14个调度器 + Checker)
```

# 9. 测试与验证

## 9.1 测试覆盖

官方测试用例：

| 示例 | 描述    | 预期 Fin | 实际 Fin | 状态 |
|----|-------|--------|--------|----|
| 1  | 基础场景  | 12040  | 12040  | ✓  |
| 2  | 并行优化  | 12020  | 12020  | ✓  |
| 3  | 同任务并行 | 13020  | 13020  | ✓  |

自研测试集 (test\_cases.json)：

编写了17个测试用例，覆盖9大类场景，全面验证算法的正确性和性能：

9.1.1 基础功能测试 (3个)

| 测试用例  | 描述                 | 预期Fin | 验证点                      |
|-------|--------------------|-------|--------------------------|
| 基础测试1 | 两个请求顺序执行，需要offload | 12040 | 基本Reload/Offload/Visit流程 |
| 基础测试2 | 三个请求混合，不同时间开始      | 12020 | 时间依赖关系                   |
| 基础测试3 | 两个请求同时开始           | 13020 | 组级别Visit对齐               |

9.1.2 压力与边界测试 (4个)

| 测试用例  | 描述                | 预期Fin  | 验证点      |
|-------|-------------------|--------|----------|
| 压力测试1 | 5个请求同时开始          | 12000  | 组级别处理能力  |
| 压力测试2 | HBM容量不足，频繁offload | 40400  | 内存压力下的调度 |
| 边界测试1 | 恰好装满HBM，无需offload | 8100   | 容量边界处理   |
| 时间测试2 | 大时间间隔（100000单位）   | 201000 | 长时间跨度调度  |

9.1.3 算法核心验证 (5个)

| 测试用例      | 描述               | 预期Fin | 验证点            |
|-----------|------------------|-------|----------------|
| Bélády测试1 | 最优替换策略验证         | 22400 | 未来最晚使用策略       |
| 复杂测试1     | 重叠内存区域           | 8700  | 部分卸载优化         |
| 复杂测试4     | 多级部分卸载（6请求交错）    | 67000 | 复杂场景下的部分卸载     |
| 复杂测试5     | 小区间部分卸载与再利用（7请求） | 55800 | 精细化内存管理        |
| 波前测试1     | 局部切片卸载           | 22000 | Wavefront调度器验证 |

9.1.4 并行与活跃保护 (5个)

| 测试用例  | 描述                  | 预期Fin | 验证点       |
|-------|---------------------|-------|-----------|
| 并行测试2 | 多Visit同时执行（800时间单位） | 21200 | Visit并行优化 |

| 测试用例    | 描述          | 预期Fin | 验证点          |
|---------|-------------|-------|--------------|
| 活跃保护测试3 | 长短任务交错（6请求） | 61000 | 活跃锁定机制       |
| 复杂测试5   | 交错保护与再利用    | 56600 | BTreeMap高效清理 |
| 时间测试1   | 极早开始时间（t=0） | 12200 | 最早Reload时机   |
| 极限测试2   | 8个同时开始      | 33500 | 大规模组处理       |

9.1.5 极限场景（2个）

| 测试用例  | 描述                | 预期Fin | 验证点    |
|-------|-------------------|-------|--------|
| 极限测试1 | 10个小请求顺序执行        | 20100 | 多请求序列化 |
| 极限测试2 | 8个请求同时开始（800 HBM） | 33500 | 极限并发处理 |

测试覆盖统计：

- **总用例数**：20个（官方3个 + 自研17个）
- **场景覆盖**：基础功能、压力测试、边界条件、算法核心、并行优化、活跃保护、极限场景
- **复杂度**：最大10请求，时间跨度达100000单位，多级交错访问
- **通过率**：100%（所有用例通过Checker验证）

JSON测试框架示例：

```
1 {
2 "tests": [
3 {
4 "name": "基础测试1 - 简单顺序",
5 "description": "两个请求顺序执行，需要offload",
6 "input": "200 100 2\n0 100 0 30\n100 100 50 10",
7 "expected_fin": 12040
8 },
9 {
10 "name": "压力测试1 - 多个同时请求",
11 "description": "5个请求同时开始，测试组级别处理",
12 "input": "500 300 5\n0 50 0 2000\n50 50 0 2000...",
13 "expected_fin": 12000
14 },
15 {
16 "name": "Bélády测试1 - 最优替换策略",
17 "description": "测试基于未来使用的最优替换算法",
18 "input": "300 150 4\n0 100 0 100\n100 100 500 100...",
19 "expected_fin": 22400
20 }
21],
22 "metadata": {
23 "version": "1.0",
```

```

24 "total_cases": 17,
25 "categories": {
26 "基础测试": 3,
27 "压力测试": 2,
28 "边界测试": 2,
29 "复杂测试": 5,
30 "时间测试": 2,
31 "Bélády测试": 2,
32 "并行测试": 2,
33 "活跃保护测试": 3,
34 "极限测试": 2,
35 "波前测试": 1
36 }
37 }
38 }

```

**运行命令：**

```

1 cargo run test # 官方测试（3个）
2 cargo run test-json # JSON扩展测试（17个）

```

## 9.2 正确性验证

**Checker 工具：**

```

1 ./checker/checker infile.txt outfile.txt expected.txt

```

**验证项：**

1. ✓ 输出格式正确
2. ✓ 时间约束满足
3. ✓ 内存容量不超限
4. ✓ Visit 期间内存在 HBM
5. ✓ 操作顺序合法
6. ✓ 同组请求 Visit 对齐

**Dispatcher 集成 Checker：**

```

1 // 验证每个调度器的输出
2 if checker_enabled {
3 let valid = run_checker(&input, &output);
4 if !valid {
5 eprintln!(" X Checker 验证失败");
6 continue;
7 }
8 eprintln!(" ✓ Checker 验证通过");
9 }

```

**所有测试用例通过 Checker 验证。**

# 10. 总结与展望

## 10.1 核心贡献

1. 多策略调度框架：
- 实现了14种不同调度策略

■ Dispatcher 自动选择最优结果

■ 基于 Trait 的可扩展架构
2. 算法设计：
- Bélády 最优页面替换算法

■ 组级别 Visit 对齐机制

■ 部分卸载优化策略

■ 完善的活跃锁定机制
3. 工程实现：
- 模块化代码结构（~3200行）

■ 高效的数据结构选择

■ 完善的测试框架（JSON + Checker）

■ 详细的日志输出（GMP\_VERBOSE）
4. 性能表现：
- 官方3个示例均达到最优结果

■ 执行时间 < 0.5 秒

■ 内存占用 < 50 MB

## 10.2 优势分析

| 方面   | 优势                  |
|------|---------------------|
| 理论基础 | Bélády 算法的理论最优性     |
| 实用性  | 多策略自动选择，适应不同场景      |
| 扩展性  | Trait 设计，易于添加新策略    |
| 鲁棒性  | 处理各种边界情况，Checker 验证 |
| 可维护性 | 模块化结构，代码清晰          |

## 10.3 项目结构

```
1 GMP_for_LLM/
2 |— src/
3 | |— main.rs # 入口程序
4 | |— lib.rs # 库入口
5 | |— types.rs # 数据类型定义
6 | |— memory.rs # 内存管理工具
7 | |— scheduler_trait.rs # Scheduler Trait
8 | |— dispatcher.rs # 调度器选择器
```

```

9 | | └─ schedulers/
10 | | └─ mod.rs # 模块导出
11 | | └─ greedy_scheduler.rs # Bélády
12 | | └─ wavefront_scheduler.rs # 波前
13 | | └─ lru_scheduler.rs # LRU
14 | | └─ aggressive_prefetch_scheduler.rs
15 | | └─ lazy_scheduler.rs
16 | | └─ base_scheduler.rs
17 | | └─ slidingwindowgreedy_scheduler.rs
18 | | └─ overlap_aware_eviction_scheduler.rs
19 | | └─ cost_benefit_scheduler.rs
20 | | └─ lfu_scheduler.rs
21 | | └─ fragmentation_scheduler.rs
22 | | └─ just_in_time_scheduler.rs # JIT调度
23 | | └─ generational_scheduler.rs # 分代调度
24 | └─ test_cases.json # 测试用例
25 | └─ Cargo.toml # 项目配置
26 | └─ README.md # 使用说明
27 | └─ ALGORITHM_DESIGN.md # 算法文档

```

## 10.4 改进方向

短期优化：

### 1. 预计算优化：

```

1 // 预计算所有区域的下次使用时间
2 let next_use_cache: HashMap<i64, i64> =
 precompute_next_use(&reqs);

```

### 2. 并行调度器执行：

```

1 use rayon::prelude::*;
2
3 let results: Vec<_> = schedulers.par_iter()
4 .map(|s| s.schedule(&reqs, 1, m))
5 .collect();

```

长期改进：

### 1. 机器学习选择器：

- 根据输入特征（N, M, L, 重叠度等）预测最优策略
- 避免运行所有调度器

### 2. 在线学习：

- 支持动态请求（streaming）
- 实时调整策略

## 10.5 实际应用展望

本方案可应用于：

- 1. **大模型训练：**
  - 优化 GPU HBM 与 DRAM 之间的数据传输
  - 减少训练时延
- 2. **大模型推理：**
  - 动态批处理场景下的内存管理
  - 提高吞吐量
- 3. **数据库系统：**
  - 缓冲池管理
  - 查询优化
- 4. **操作系统：**
  - 虚拟内存页面替换
  - 改进传统 LRU/FIFO

## 10.6 结语

本项目通过将经典的 Bélády 算法与现代 LLM 内存管理需求相结合，设计了一个**高效、可靠、可扩展**的全局内存调度方案。我们的实现在保证正确性的前提下，通过多策略框架在不同测试场景中都能找到最优或接近最优的调度方案。

**核心创新点：**

- ☒ 多策略自动选择框架
- ☒ 部分卸载优化
- ☒ 完善的活跃锁定

**成果总结：**

- ☒ 官方测试 100% 通过
- ☒ Checker 验证全部合格
- ☒ 代码结构清晰可维护
- ☒ 性能达到预期目标

未来，我们将继续优化算法，特别是在大规模并发场景下的表现，并探索将该方案应用于实际的 LLM 训练和推理系统中，为大模型的高效运行提供强有力的内存管理支持。

## 附录

### A. 完整代码统计

|   |                        |
|---|------------------------|
| 1 | 总代码行数：~3500行           |
| 2 | ├─ 核心调度逻辑：~2100行       |
| 3 | ├─ 数据结构和工具：~600行       |
| 4 | ├─ Dispatcher 框架：~400行 |
| 5 | └─ 测试和验证：~400行         |
| 6 |                        |



```
7 平均每个调度器：~150行
8 调度器总数：14个
```

## B. 运行指南

```
1 # 终端交互输入输出
2 cargo run
3
4 # 文件输入输出
5 cargo run < infile.txt > outfile.txt
6
7 # 运行官方示例测试
8 cargo run test
9
10 # 运行JSON扩展测试
11 cargo run test-json
12
13 # 详细日志模式
14 GMP_VERBOSE=1 cargo run < infile.txt
15
16 # 指定调度器（开发调试用）
17 GMP_SCHEDULER=GreedyScheduler cargo run < infile.txt
18
19 # 运行 checker 验证
20 ./checker/checker infile.txt outfile.txt expected.txt
```

## C. 依赖说明

```
1 [package]
2 name = "gmp_for_llm"
3 version = "0.1.0"
4 edition = "2021"
5
6 [dependencies]
7 serde = { version = "1.0", features = ["derive"] }
8 serde_json = "1.0"
```

依赖用途：

- `serde`: 用于JSON 序列化/反序列化
- `serde_json`: JSON 测试框架支持

## D. 环境要求

```
1 操作系统: Ubuntu 24.04.2 LTS（推荐）或其他 Linux 发行版
2 编译器: rustc 1.91.0+
3 构建工具: cargo 1.91.0+
```

## E. 联系方式

项目地址: [https://github.com/guohuan78/GMP\\_for\\_LLM](https://github.com/guohuan78/GMP_for_LLM) (赛后开源)

邮箱: [gh2002@mail.nwpu.edu.cn](mailto:gh2002@mail.nwpu.edu.cn)

团队: 西北工业大学 - 郭皖、包子旭、沈铭

指导老师: 张羽教授

---

文档版本: v2.0 (最终版)

最后更新: 2025年11月25日

文档说明: 本文档综合了算法设计、工程实现、性能分析等多个方面, 完整描述了 GMP\_for\_LLM 项目的技术细节和创新点。